

Relatório do Trabalho Prático 3

Programação Paralela com MPI

Êxodo Melo, Thiago Aparecido

19 de Junho de 2026

Resumo

O presente relatório tem o objetivo apresentar o segundo trabalho feito sobre a disciplina de Programação Paralela de paralelização da solução numérica de sistemas lineares utilizando o Método Iterativo de Jacobi utilizando MPI e seus resultados.

1 Introdução

Sistemas lineares aparecem em diversos locais, desde circuitos elétricos e eletrônicos à sistemas de controle industriais. É de interesse a busca de métodos para solucioná-los.

Um desses métodos é o Método de Jacobi para solução de sistemas lineares, caracterizada pela atualização independente de cada componente do vetor solução a partir dos valores da iteração anterior. A paralelização desse método surge do custo de $O(N^2)$ para cada iteração; tendo independência entre atualizações do vetor solução torna o método particularmente adequado para execução paralela, permitindo explorar arquiteturas de memória compartilhada de forma eficiente.

O método de Jacobi parte da seguinte lógica:

$$\begin{aligned}\sum_j A_{ij}x_j &= b_i \\ A_{ii}x_i + \sum_{j \neq i} A_{ij}x_j &= b_i \\ A_{ii}x_i &= b_i - \sum_{j \neq i} A_{ij}x_j \\ x_i &= \frac{b_i - \sum_{j \neq i} A_{ij}x_j}{A_{ii}}\end{aligned}\tag{1}$$

Tendo uma estimativa inicial para $\mathbf{x}$, pode-se atualizar iterativamente valendo pela equação. Para a convergência do método é precisa que a matriz $\mathbf{A}$ seja uma matriz de diagonal dominante, isto é:

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}|, \forall i$$

A paralelização é um processo difícil e dependente de máquina, uma das maneiras de facilitar a implementação é a abstração de detalhes menos importantes para o programador. MPI é uma dessas abstrações, qual permite utilizar a arquitetura paralela de conjuntos de processadores para executar programas computacionalmente intensivos.

Neste trabalho, é apresentada uma implementação do método de Jacobi utilizando MPI, com o objetivo de explorar o paralelismo de dados e avaliar o impacto da paralelização no desempenho do algoritmo.

É analisado a escolha de número de processos e como organizar as regiões de computação para sua implementação em MPI, além de detalhes de sincronização, concorrência e comunicação agora diretamente gerenciados pelo programador.

Dessa forma, o trabalho busca não apenas demonstrar a aplicação prática de técnicas de paralelização, mas também discutir as limitações e desafios associados à obtenção de ganho de desempenho com a solução paralela.

A implementação com OpenMPI compartilha sua lógica de paralelismo com OpenMP e Cuda, todas explorando paralelismo de dados. Em termos de complexidade de programação, pode-se dizer que a sequência OpenMP, Cuda e OpenMPI segue uma escada: OpenMP sendo a mais simples, tendo sincronizações e compartilhamento de dados completamente abstraídos; Cuda logo em seguida, sendo necessário lidar com sincronizações e uma pequena porção de comunicação de dados; e então OpenMPI, qual precisa lidar com sincronização e comunicação de dados entre diferentes processos.

2 Ambiente Experimental

Os experimentos foram executados no ambiente Google Colab, utilizando o compilador `mpic++.openmpi` (G++ 11.4.0) e biblioteca MPI (OpenMPI 4.1.2). O código sequencial foi compilado com `g++`. Foi utilizado o parâmetro de compilação `-O3` para otimização pelo compilador.

Foram gerados sistemas lineares consistentes de ordem 1024×1024 , com valores aleatórios para $\mathbf{A}$ e $\mathbf{x}$, garantindo convergência do método de Jacobi. O critério de parada utilizado foi $\epsilon = 0.0001$, com número máximo de 100 iterações, de forma a gerar dados de tempo comparáveis com os trabalhos anteriores.

Para cada configuração de número de processos, foram realizadas 5 execuções independentes, a fim de reduzir variações de desempenho causadas pelo escalonamento de processos e processos concorrentes.

O tempo de execução foi medido utilizando um próprio do OpenMPI (`MPL_Wtime()`). O início da medição ocorre imediatamente antes da primeira iteração do algo-

ritmo, após o envio das matrizes A e B, enquanto o término é registrado após a finalização do laço iterativo no processo principal.

É feito o experimentos com números de processos entre 2 e 64, números retirados do Trabalho 01 com OpenMP, com a expectativa de um comportamento similar.

3 Implementação

O algoritmo implementado corresponde ao método iterativo de Jacobi para resolução de sistemas lineares, no qual o vetor solução é atualizado iterativamente a partir dos valores da iteração anterior. A paralelização foi realizada utilizando MPI, explorando o paralelismo no nível de dados presente nas iterações independentes do método.

A implementação MPI do método de Jacobi foi desenvolvida explorando o paralelismo de dados presente no algoritmo, onde um grupo de equações do sistema linear é distribuída a um processo. Dessa forma, cada processo é responsável pelo cálculo de um grupo do vetor solução na iteração atual.

A matriz é particionada igualmente, tendo para cada processo, a linha da matriz dos itens que é responsável o cálculo.

Em cada processo é realizado o cálculo para o grupo de itens na seguinte forma:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} A_{ij} x_j^{(k)}}{A_{ii}}$$

O carregamento dos arquivos de entrada é feito pelo processo principal, após o sucesso, é feito *broadcast* do tamanho N do sistema. Caso o tamanho N do sistema não seja múltiplo do número de processos, é finalizado, sendo essa uma restrição imposta para simplificar a lógica de comunicação.

Foi utilizando a ordem de 1024 para os experimentos para permitir a divisão pelos números de processos escolhidos para experimentação e ser próximo ao ordem utilizada por experimentos dos outros dois trabalhos, como forma a ter uma comparação mais equivalente entre os métodos de paralelização.

Determinado o tamanho do grupo de cada processo, é feito o *scatter* das linhas da matriz A e itens do vetor B . Cada processo, assim, possui o X , A e B dos itens que é responsável, X sendo inicializado com 0.

É utilizado o *Allgather* para coletar os X locais de cada processo e uni-los em um único X . Com os valores de X sincronizados, cada processo calcula o erro parcial de seus itens e é executado um *Allreduce* com a operação de soma.

3.1 Decisões de Projeto

O número de processos não foi fixado previamente, sendo tratado como um parâmetro experimental. Foram realizados testes variando essa configuração com o objetivo de avaliar o impacto no desempenho do algoritmo em diferentes níveis de paralelismo.

É utilizado comunicação coletiva para os dados, como uma forma de simplificar a comunicação e por necessidade do método, é necessário para o cálculo da próxima iteração de X_i conhecimento de todas outras posições.

A divisão de itens foi definida com base no ordem do sistema linear e no número de processos, garantindo cobertura completa do domínio computacional. Para isso, foi utilizada a seguinte fórmula:

$$\#linhas = \frac{N}{\#processos}$$

Essa abordagem garante a distribuição uniforme de itens sobre os processos.

O critério de parada do algoritmo é baseado em dois fatores: o número máximo de iterações e a convergência do método, medida pelo erro calculado. A execução é interrompida quando o erro é menor que um valor limite (ϵ) ou quando o número máximo de iterações é atingido.

4 Resultados

4.1 Desempenho

Os experimentos foram realizados variando o número de processos utilizadas na execução paralela do método de Jacobi. Para cada configuração, foram realizadas cinco execuções, sendo considerado o tempo médio.

O speedup foi calculado em relação à execução sequencial (1 processo), conforme a Equação:

$$S(p) = \frac{T(1)}{T(p)}$$

A eficiência foi calculada como:

$$E(p) = \frac{S(p)}{p}$$

Abaixo a Tabela 1 de dados sobre tempo, speedup e eficiência contra o programa paralelo com 1 processo.

Num. Processos	Tempo (ms)	Speedup (vs. 1 processo)	Eficiência
1	391.82	-	-
2	285.33	1.37	0.70
4	226.85	1.73	0.43
8	298.85	1.31	0.16
16	297.22	1.31	0.08
32	419.97	0.93	0.03
64	594.95	0.66	0.01

Table 1: Desempenho do algoritmo MPI variando número de processos

Abaixo, os gráficos para cada medida contra o Numero de Processos.

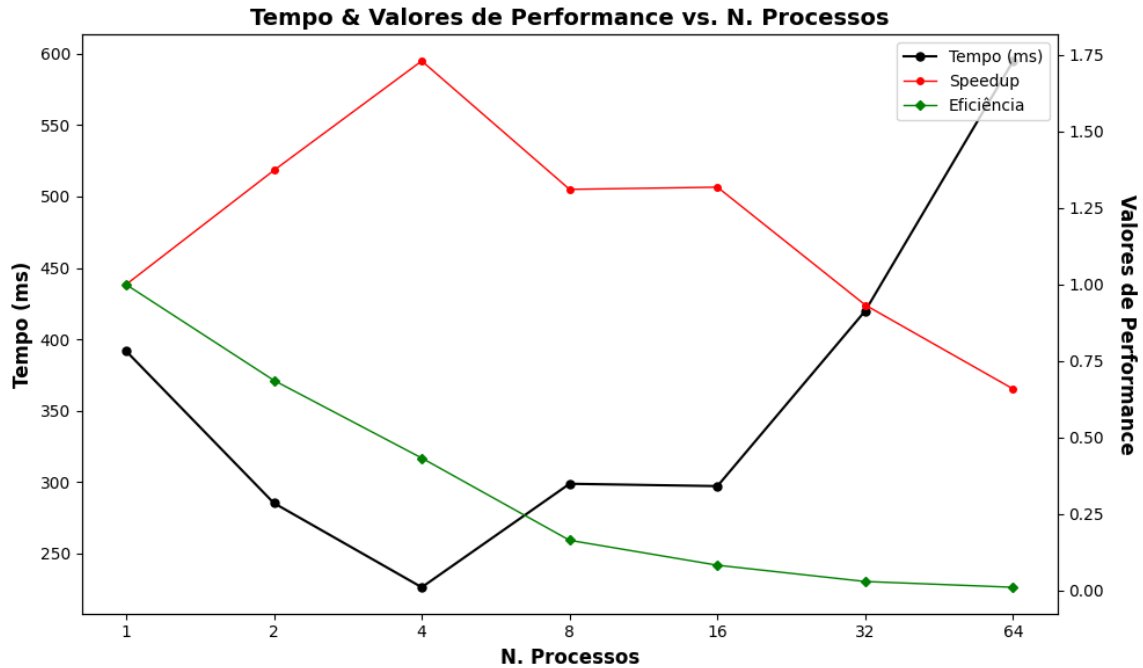


Figure 1: Tempo & Valores de Performance vs. Num. Processos

Abaixa a Tabela 2 de dados sobre speedup e eficiência contra o programa sequencial, contra o OpenMp e contra o CUDA.

Versão	Configuração	Tempo (ms)	Speedup (vs. Seq.)	Speedup (vs. OpenMp.)	Speedup (vs. Cuda.)
Seq.	-	134.634	-	-	-
OpenMP.	16 threads	6640	0.02	-	-
CUDA GPU	8 threads/bloco	6.070	22.18	1093	-
MPI	2 Processos	285.33	0.47	23.27	0.021
MPI	4 Processos	226.37	0.60	29.33	0.026
MPI	8 Processos	298.85	0.45	22.21	0.02
MPI	16 Processos	297.22	0.45	22.34	0.02
MPI	32 Processos	419.97	0.32	15.81	0.014
MPI	64 Processos	594.96	0.22	11.16	0.01

Table 2: Comparação de desempenho entre implementação sequencial, OpenMP, CUDA e MPI

Abaixo os gráficos de Speedup das implementações anteriores contra a implementação em MPI variando o numero de processos.

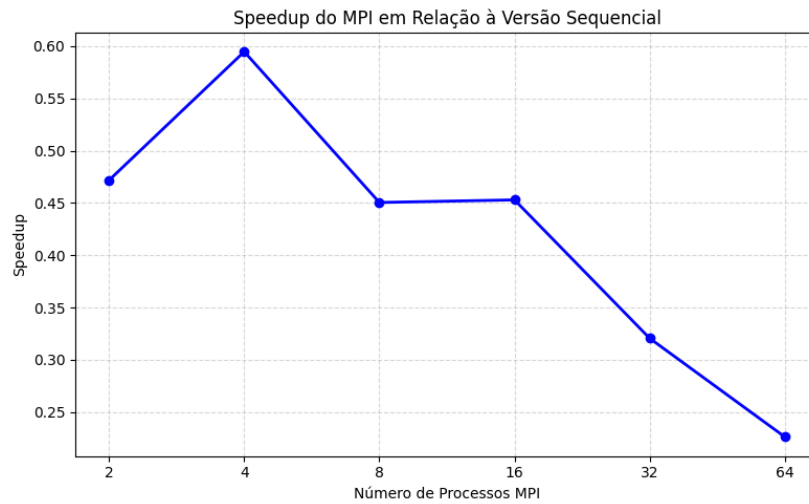


Figure 2: Speedup vs. Num. Processos (Sequencial)

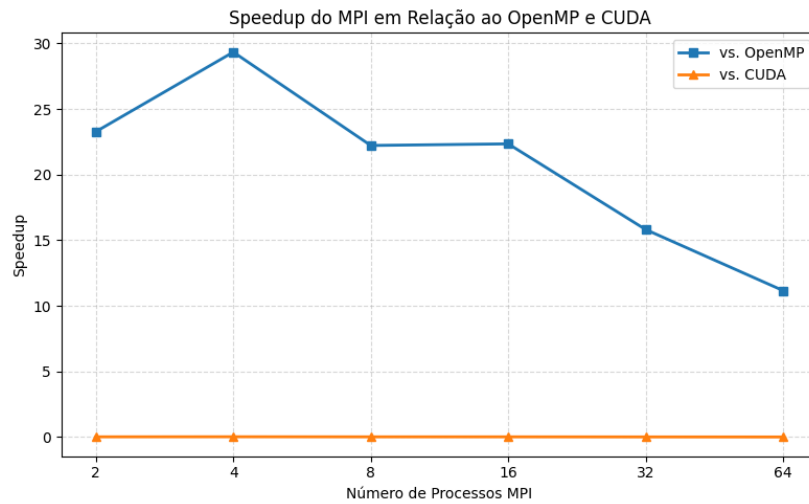


Figure 3: Speedup vs. Num. Processos (Cuda e OpenMP)

Considerando os dados da primeira tabela, utilizando a execução com um único processo MPI como referência, a melhor configuração foi obtida utilizando 4 processos, alcançando um tempo médio de execução de 226,85 ms e um speedup de 1,73. A eficiência nessa configuração foi de aproximadamente 43%, representando o melhor compromisso entre redução do tempo de execução e aproveitamento dos recursos computacionais.

Observa-se que o aumento do número de processos não proporciona ganhos lineares de desempenho. Entre 1 e 4 processos ocorre redução progressiva do tempo de execução, porém, a partir de 8 processos, o desempenho passa a se degradar. Os tempos aumentam para 298,85 ms, 297,22 ms, 419,97 ms e 594,95 ms para 8, 16, 32 e 64 processos, respectivamente.

Consequentemente, o speedup diminui e chega a valores inferiores a 1 para 32 e 64 processos, indicando que essas configurações são mais lentas que a execução com apenas um processo. Esse comportamento está associado ao aumento do custo de comunicação e sincronização entre os processos MPI, especialmente devido às operações coletivas utilizadas pelo algoritmo, como *MPI_Allgather* e *MPI_Allreduce*.

Na comparação com as demais implementações, a versão MPI apresentou desempenho inferior à implementação sequencial utilizada como referência. Enquanto a versão sequencial executou em 134,63 ms, a melhor configuração MPI (4 processos) obteve 226,85 ms, correspondendo a um speedup de apenas 0,60 em relação à execução sequencial. Isso indica que o overhead de comunicação entre processos superou os ganhos obtidos pela paralelização para o tamanho do problema considerado.

Em relação à implementação OpenMP, todas as configurações MPI apresentaram tempos significativamente menores. A implementação OpenMP executou em 6640 ms, enquanto as versões MPI variaram entre aproximadamente 227 ms e 595 ms, resultando em speedups entre 11,16 e 29,33 vezes a favor do MPI. Há de se ter em mente que os dados de tempo de execução OpenMP utiliza uma máquina diferente, sendo OpenMPI e GPU/Cuda com máquinas do Google Colab.

Por outro lado, a implementação CUDA apresentou o melhor desempenho entre todas as versões avaliadas, com tempo de execução de apenas 6,07 ms. Comparando-se com a GPU, a implementação MPI apresentou speedups inferiores a 1 em todas as configurações, variando entre aproximadamente 0,01 e 0,03. Isso demonstra a superioridade da abordagem baseada em GPU para este problema, uma vez que o elevado grau de paralelismo disponível no hardware gráfico reduz significativamente o tempo de processamento.

De forma geral, os resultados indicam que a implementação MPI apresenta ganhos moderados quando comparada à sua execução com um único processo, alcançando o melhor desempenho com 4 processos. Entretanto, o aumento excessivo do número de processos reduz significativamente a eficiência devido ao crescimento do overhead de comunicação. Além disso, para o tamanho do problema utilizado neste trabalho, a implementação MPI não conseguiu superar a versão sequencial nem a implementação CUDA, embora tenha apresentado desempenho consideravelmente superior ao da implementação OpenMP.

4.2 Convergência

A convergência do método de Jacobi depende diretamente das propriedades da matriz de coeficientes. Em particular, uma condição suficiente para a convergência é que a matriz seja diagonalmente dominante. No entanto, nem todos

os sistemas utilizados nos experimentos satisfizeram essa propriedade.

Durante a execução dos testes, foram observados casos em que o método não convergiu. Nessas situações, o erro entre iterações não apresentou comportamento decrescente e, em alguns casos, cresceu progressivamente ao longo das iterações. Esse comportamento pode levar à divergência numérica, fazendo com que os valores do vetor solução assumam magnitudes muito elevadas, resultando em valores infinitos (`inf` ou `-inf`).

O critério de parada adotado foi baseado na norma Euclidiana da diferença entre iterações consecutivas:

$$\|X^{(k+1)} - X^{(k)}\|_2$$

Nos casos em que o método converge, esse erro tende a zero à medida que o número de iterações aumenta. Por outro lado, nos casos divergentes, o erro tende a crescer ou oscilar, impedindo que o critério de parada baseado em ϵ seja satisfeito.

Além disso, é importante destacar que a paralelização do algoritmo não influencia a convergência do método, pois não altera sua formulação matemática. A divergência observada está ligada exclusivamente às características do sistema linear e não à utilização de múltiplas threads.

Dessa forma, conclui-se que a garantia de convergência do método de Jacobi depende fortemente da construção da matriz de coeficientes, sendo fundamental propriedades como dominância diagonal para aplicações práticas.

É possível realizar alterações na forma qual a tolerância é disposta a depender do número de threads. Entretanto, não realizado tal implementação nesse trabalho, mantendo a tolerância igual para todas threads.

Abaixo temos a uma Tabela 3 iterações vs erro para a execução do algoritmo de um sistemas de ordem 1000, número de iterações 300 e $\epsilon = 0.0001$ sendo o sistema gerado convergente.

Iteração	Erro (Sistema 01)
0	322,77
50	304,22
100	298,2
200	286,49
299	275,36

Table 3: Erro dos sistemas para cada iteração

4.3 Discussão

Os resultados obtidos demonstram que a utilização de MPI proporcionou redução do tempo de execução em relação à execução do MPI com apenas um processo. A melhor configuração observada foi utilizando 4 processos, alcançando tempo médio de execução de 226,85 ms e speedup de 1,73 em relação à configuração com 1 processo, que apresentou tempo médio de 391,82 ms.

O método de Jacobi apresenta características favoráveis à paralelização, pois o cálculo de cada componente do vetor solução em uma determinada iteração depende apenas dos valores obtidos na iteração anterior. Dessa forma, foi possível dividir as linhas da matriz entre os processos MPI, permitindo que cada processo realizasse parte dos cálculos de forma concorrente.

Entretanto, os resultados mostram que o aumento do número de processos não proporciona ganhos lineares de desempenho. Após a configuração com 4 processos, observa-se uma redução significativa da eficiência paralela. Embora os tempos para 8 e 16 processos permaneçam inferiores ao obtido com apenas um processo, o desempenho deixa de melhorar e passa a se degradar.

Para 32 e 64 processos, os tempos de execução tornam-se superiores ao da configuração de referência, resultando em speedups inferiores a 1. Esse comportamento ocorre devido ao aumento do custo de comunicação e sincronização entre os processos, principalmente pelas operações coletivas utilizadas durante a execução do algoritmo, como `MPI_Allgather` e `MPI_Allreduce`.

A redução da eficiência com o aumento do número de processos indica que, para o tamanho do problema analisado, o custo de gerenciamento e comunicação entre os processos passa a representar uma parcela significativa do tempo total de execução. Dessa forma, o aumento do paralelismo deixa de compensar a sobrecarga introduzida pela comunicação.

Na comparação com as demais implementações, a versão MPI apresentou desempenho superior à implementação OpenMP em todas as configurações avaliadas. A melhor configuração MPI (4 processos) executou aproximadamente 29 vezes mais rápido que a versão OpenMP. Por outro lado, a implementação CUDA apresentou o melhor desempenho geral, com tempo de execução de apenas 6,07 ms. Dessa forma, todas as configurações MPI apresentaram desempenho inferior à implementação em GPU, evidenciando a elevada capacidade de paralelização oferecida pela arquitetura CUDA para este tipo de problema.

Além disso, a implementação sequencial apresentou tempo de execução de 134,63 ms, valor inferior ao obtido por qualquer configuração MPI. Esse resultado sugere que, para o tamanho do problema utilizado neste trabalho, o overhead associado à comunicação entre processos supera os benefícios da paralelização distribuída.

De maneira geral, os experimentos mostram que a implementação MPI apresenta escalabilidade limitada para o problema analisado. Embora seja possível obter ganhos em relação à execução MPI com um único processo, o aumento excessivo do número de processos reduz significativamente a eficiência devido ao custo das operações de comunicação. Os resultados indicam que a configuração com 4 processos representa o melhor equilíbrio entre paralelismo e overhead de comunicação no ambiente avaliado.

5 Conclusão

Neste trabalho foi desenvolvida uma implementação paralela do método iterativo de Jacobi utilizando OpenMPI, explorando o paralelismo por meio da

distribuição das linhas da matriz entre múltiplos processos.

Os resultados experimentais mostraram que a implementação MPI apresentou seu melhor desempenho utilizando 4 processos, alcançando tempo médio de execução de 226,85 ms. Entretanto, quando comparada à implementação sequencial, que executou em 134,63 ms, a versão MPI não foi capaz de obter ganho de desempenho, apresentando speedup máximo de apenas 0,60 em relação à execução sequencial.

Também foi observado que o aumento do número de processos não resultou em ganhos proporcionais de desempenho. Embora a execução com 2 e 4 processos tenha reduzido o tempo em relação à configuração MPI com um único processo, a partir de 8 processos o desempenho passou a se degradar gradualmente. Para 32 e 64 processos, o tempo de execução tornou-se significativamente maior, evidenciando o impacto do custo de comunicação e sincronização entre os processos.

Na comparação com as demais implementações, a versão MPI apresentou desempenho significativamente superior ao OpenMP utilizado neste trabalho, mas inferior tanto à implementação sequencial quanto à implementação CUDA. A versão CUDA obteve o melhor resultado geral, com tempo de execução de apenas 6,07 ms, demonstrando a elevada eficiência da arquitetura GPU para o processamento paralelo do método de Jacobi.

Dessa forma, conclui-se que a utilização de MPI, para o tamanho do problema analisado e para o ambiente de execução utilizado, não proporcionou ganhos de desempenho em relação à implementação sequencial. Os resultados indicam que o overhead associado às operações de comunicação entre processos teve impacto significativo no tempo total de execução. Como trabalho futuro, recomenda-se avaliar instâncias maiores do problema, nas quais o custo computacional possa superar o overhead de comunicação, permitindo um melhor aproveitamento do paralelismo distribuído.

6 Referencias

- [1] Vipin Kumar, Ananth Grama, Anshul Gupta, and George Karypis. *Introduction to parallel computing*, volume 110. Benjamin/Cummings Redwood City, CA, 1994.
- [2] Philip Wilkinson. *Parallel programming: Techniques and applications using networked workstations and parallel computers*, 2/E. Pearson Education India, 2006.